

Organização de Computadores 2024/25

Laboratório 1 – Group 06

(ist 106035, ist 1105849, ist 1107294)

4.1 Directly-Mapped L1 Cache

We used SimpleCache.c as a starting point for our project. In our implementation, we used shifts and masks to extract the tag (31-14), the index (13-6) and the offset (5-0). We could have done the same thing using only divisions and mod and the macros from Cache.h but we believe it is easier to read our implementation.

In order to develop a direct-mapped cache with several lines, we added to the Cache struct an array of CacheLines and to calculate the number of lines we used $L1_SIZE / BLOCK_SIZE$.

To store the information in a line we created a Data[BLOCK_SIZE] array within the struct CacheLine.

Since now each block has 16 words we use offset to identify which word we're looking for.

4.2 Directly-Mapped L2 Cache

In this task, both caches are similar to the L1 cache from the previous task. The only changes are the tag bits (31-15) and the index bits (14-6) for L2 and the fact that now, when there is a miss on the L1 cache, we first access the L2 cache and we only access the DRAM if we also have a miss on the L2 cache.

4.3 2-Way L2 Cache

This task has the most significant code changes. To calculate the number of sets (L2_SETS) we used $L2_SIZE / (BLOCK_SIZE) * L2_WAYS$.

For the L2 cache the tag bits are 31-15 and the index bits are 14-6. We could easily conclude this because there are 512 blocks and sets with size 2 which means we will have $512/2 = 256$ sets and 8 bits of index to map all the 2^8 sets.

We decided to use an array of arrays for the L2 cache (`lines[L2_SETS][L2_WAYS]`) and now each cache line has an `uint32_32` LRU variable that stores the last time the line was used. To find the last recently used line we compare the LRU variable from both lines and choose the one that has the lower value.

The major code change we see is in the `accessL2` function. Now we have to check for a cache hit in both line blocks. If none of them have a hit, we choose the least recently used one. Every time we write to a L2 block we update the LRU variable by using the `getTime()` function.

Tests

We also created a test for our program: `Teste_alternados.c`

It is a simple program with the purpose of comparing the difference between the response time of all three cache systems. There are four instructions in the test:

- Read address 20864;
- Read address 53632;
- Read address 20864;
- Read address 53632.

The addresses used in this test (20864 and 53632) have the same index bits on all three cache systems which means they will be mapped to the same block (in the direct-mapped case) and to the same set (in the 2-Way Cache case).

In the Directly-Mapped L1 Cache, there will be four misses, that means we'll have to access the DRAM four times. This means that the time spent will be $4 * (\text{DRAM_READ_TIME} (100) + \text{L1_READ_TIME} (1))$, obtaining a total of 404.

In the Directly-Mapped L2 Cache, even with a two-level cache system, we'll still have four misses which means that the time spent will be the sum of `DRAM_READ_TIME` (100), `L2_READ_TIME` (10) and `L1_READ_TIME` (1), all multiplied by 4, which gives us a total of 444.

Finally, in the 2-Way L2 Cache the first two reads that we execute will be a miss (compulsive cache failure since the cache is empty). After that we will only have hits because, although the addresses are mapped to the same set, there will not be a conflict failure as there are two entries in each set. This means that the time spent will be $2 * (\text{DRAM_READ_TIME} (100) + \text{L2_READ_TIME} (10) + \text{L1_READ_TIME} (1)) + 2 * (\text{L2_READ_TIME} (10) + \text{L1_READ_TIME} (1))$, that's equal to 244.